

## AUTHOR QUERIES

### AUTHOR PLEASE ANSWER ALL QUERIES

**PLEASE NOTE:** We cannot accept new source files as corrections for your paper. If possible, please annotate the PDF proof we have sent you with your corrections and upload it via the Author Gateway. Alternatively, you may send us your corrections in list format. You may also upload revised graphics via the Author Gateway.

Carefully check the page proofs (and coordinate with all authors); additional changes or updates **WILL NOT** be accepted after the article is published online/print in its final form. Please check author names and affiliations, funding, as well as the overall article for any errors prior to sending in your author proof corrections.

AQ1: Please confirm or add details for any funding or financial support for the research of this article.

AQ2: Please check whether the equal contribution footnote is correct as set.

AQ3: Please provide the department name for Tianmushan Laboratory and Zhongguancun Laboratory.

AQ4: Current affiliation in the biography of Jian Mao does not match the First Footnote. Please check and correct where needed.

# Anomaly Detection in Smart IoT Systems Based on Contextual Semantics of Behavior Graphs

Qixiao Lin<sup>1b</sup>, Shuyuan Chang, Jian Mao<sup>1b</sup>, *Member, IEEE*, Qiange Liu, Ziwen Liu<sup>1b</sup>,  
Yifan Lu, and Yan Huo<sup>1b</sup>, *Senior Member, IEEE*

**Abstract**—With the advancement of Internet of Things (IoT) technology, smart IoT systems have become integral to industrial production and daily life. However, they face significant security and privacy vulnerabilities from different aspects. To enhance the security mechanisms, “Meta Computing” techniques (also called “Network-as-a-Computer, NaaC”) integrate all available computing resources and support zero-trust environments. Traditional anomaly detection methods consider the correlation between two events, which can be bypassed by constructing fake events that indirectly influence target devices, leading to false negatives. To address this issue, behavior-context-based approaches struggle with the complexity and variability of behavior patterns, resulting in false positives due to their inability to tolerate slight differences in event sequences representing the same system behavior. In this article, we propose an anomaly detection approach in smart IoT systems based on the contextual semantics of behavior graphs. Our method captures critical event semantics while tolerating variations in noncritical events to aggregate and summarize the behavior semantics. We cluster benign behaviors and use whether the testing behavior instance falls into the benign behavior clusters as the criterion for anomaly detection. Our experiment results show that our approach effectively differentiates between anomalous and benign behaviors, significantly reducing false positives and negatives compared to state-of-the-art methods.

**Index Terms**—Anomaly detection, Internet of Things (IoT), meta computing, system behavior analysis.

## I. INTRODUCTION

WITH the development of Internet of Things (IoT) technology, smart IoT systems are gradually integrated into daily life, significantly enhancing the convenience and efficiency of production and living, e.g., the Industrial IoT, Smart Home, Smart City, etc. Various sensors, controllers, and

automated devices deployed throughout the smart IoT system allow users to manage and control devices remotely, achieving highly intelligent management. However, smart IoT systems have numerous security and privacy vulnerabilities regarding physical devices [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], network protocols [11], [12], [13], [14], [15], [16], application platforms [17], [18], [19], [20], [21], and so on, threatening users’ property, privacy, and even physical safety. For example, in an IIoT system, attackers can infect the Web application hosted by the emerging embedded web servers within the PLCs and modify the controls to damage physical machines [21]. Specifically, they exploited an arbitrary file upload vulnerability leveraging a CORS misconfiguration vulnerability in conjunction with an authentication bypass vulnerability, which overrode the original PLC GUI file on the WAGO 750’s firmware with a duplicate page that contained malicious code.

In order to promptly identify and prevent harmful behaviors in smart IoT systems, researchers are attempting to integrate all available computing resources and enhance system security comprehensively through the visualized Network-as-a-Computer, that is, meta computing [22], [23]. In addition to authorization/access control [24], [25], [26], [27], [28], [29], [30] and security/safety policy enforcement [31], [32], [33], anomaly detection is also a common security enhancement mechanism. In the context of heterogeneous smart IoT systems with diverse event log sources, we adopt the paradigm of Meta Computing to integrate computing resources across the cloud, edge, and end devices. This paradigm enables us to collect logs from multiple sources, thereby breaking down the barriers of heterogeneity imposed by device manufacturers and service providers. Consequently, it offers more comprehensive and user-friendly computing power and data support for anomaly detection. Early exploration in this field [34], [35], [36] focused on extracting the correlation between two events to determine whether the current event should follow the preceding event. However, advanced attackers can construct fake events, using automation rules or physical environmental effects to influence target devices indirectly. For example, the event-correlation-based methods would detect an attacker who directly controls the bathroom window to turn on. However, suppose the attacker uses cyber-attack methods to inject events of “faucet turned on” and “humidity increased” continuously. In that case, it will cause an automation rule “if the bathroom humidity rises, open the bathroom window” to be triggered, and subsequently open the window. In such a scenario, the correlation between events in this

AQ1 Received 10 October 2024; revised 17 December 2024; accepted 26 December 2024. This work was supported in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LZ23F020013, and in part by the National Natural Science Foundation of China under Grant 62172027 and Grant U24B20117. (Qixiao Lin and Shuyuan Chang contributed equally to this work.) (Corresponding author: Jian Mao.)

AQ2 Qixiao Lin, Shuyuan Chang, Qiange Liu, Ziwen Liu, and Yifan Lu are with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China.

AQ3 Jian Mao is with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China, also with Tianmushan Laboratory, Hangzhou 311121, China, also with the Hangzhou Innovation Institute, Beihang University, Hangzhou 310000, Zhejiang, China, and also with Zhongguancun Laboratory, Beijing 100083, China (e-mail: maojian@buaa.edu.cn).

Yan Huo is with the School of Electronics and Information Engineering, Beijing Jiaotong University, Beijing 100044, China.

Digital Object Identifier 10.1109/JIOT.2024.3523998

“faucet turned on—humidity increased—window open event sequence is reasonable, so it can bypass the event-correlation-based detection, causing false negatives.

To address this problem, current research [37], [38], [39] extract and learn historical system behaviors (i.e., system behavior patterns represented by a sequence of events) rather than the correlation between two events and predict future actions. If the actual event does not match the prediction, it is considered abnormal. For the attack example above, although the continuous event sequence of faucet turned on—humidity increased—window open is reasonable, they are not in the appropriate environmental context. From the *high-level* perspective of system “behavior,” it is abnormal detected by current behavior-context-based approaches.

However, system behavior patterns in smart IoT systems are complex and variable, and multiple similar (but not identical) event sequences (called system behavior instances) may represent the same system behavior. For example, for the “enter home” behavior, its event sequences could be “door open—entrance hall light on—shoe cabinet open—shoe cabinet close—door close,” or “door open—door close—shoe cabinet open—shoe cabinet close.”<sup>1</sup> These behavior-context-based approaches are insufficient to cope with variable system behavior patterns. When a new behavior instance emerges, existing methods tend to decide it as an anomaly if it does not *perfectly* match existing system behavior instances, causing false positives.

Representing system behaviors at a high level is a potential approach to mitigate the issues above. We define the *semantics* of system behavior as an indicator that describes what occurred within the system during a specific period. By extracting a high-level representation of behaviors and clustering semantically similar ones, analysts only need to inspect and label representatives from these clusters. Any deviation from benign behavior will then be efficiently flagged for attack validation.

While high-level behavior representation sounds promising, two main challenges arise when extracting behaviors and describing their semantics: 1) event semantic differentiation and 2) behavior semantic summarization. Events in smart IoT systems record the state changes of different low-level devices and lack knowledge of high-level semantics. A single event, such as “light turned on,” can represent different semantics in different scenarios. The first challenge is inferring and representing a specific event’s semantics in different scenarios (C1). Meanwhile, we observe that different events in a system behavior instance impact its whole semantics differently. Taking the “enter home” behavior as an example, the “door open” event is the most critical for the whole behavior semantic; in comparison, the occurrence or ordering of “light turned on/off” and “motion sensor on/off” is less important. To address the false positives brought by behavior-context-based approaches, we expect the behavior semantics to capture the key semantics from the critical events and tolerate minor changes in noncritical events. The second challenge

is measuring the importance of different events to a specific system behavior and aggregating events based on the “importance” weight to describe the behavior’s semantics (C2).

To address the first challenge (C1), our first key insight is that the *semantic* of an event is defined by its role played in the specific behavior and can be revealed from the context, that is, the multiple relationships among its nearby events (e.g., triggered by automation rules and affected by the physical environment). We design a graph-based behavior representation to capture events’ semantics based on their neighborhood relationship on the graph. As for the second challenge (C2), based on Shannon’s information theory [40], the information content of an event is inversely proportional to the probability of its occurrence, that is, an event with a low probability of occurrence provides a more significant amount of information upon happening. Similarly, in the smart IoT system, our intuition is that events with low-occurrence frequency are more significant in representing the system behavior semantics. We use the frequency of events to measure the importance of different events.

In this article, we present an anomaly detection approach in smart IoT systems through the contextual semantics of behavior graphs. To address the false negatives caused by event-correlation-based detection, we use the overall semantics of system behaviors for anomaly detection. We design a graph-based representation to represent different features of behaviors and use the translation-based embedding model to capture events’ semantics based on their neighborhood relationship on the graph. Based on the frequency of different events as their importance weight, we aggregate the events’ semantic representation to derive the semantic representation of behavior instances. To further generalize our approach’s tolerance for benign behavior instances, we cluster benign behavior instances and use whether the testing behavior instance falls into the benign behavior clusters as the criterion for anomaly detection. We prototype and evaluate our approach on our self-developed hybrid smart IoT testbed. The experiment results show that our approach effectively separates anomalous and benign behaviors. The V-measure of our system behavior clustering results can reach up to 0.94. Compared to the state of the art [39], our approach significantly reduces the rates of false positives and false negatives and increases precision and recall by 2.6 times and 9 times, respectively.

**Contributions:** In summary, we make the following contributions in this article.

- 1) We design a graph-based representation for a system behavior to represent the complex relationship among events in the sequences. We place nodes with different environmental characteristics on separate channels to enhance the semantic meaning of the graph and reduce the effects of environmental noises.
- 2) We propose a behavior summarization and clustering approach to classify different behavior instances into behavior clusters. Our behavior summarization ensures that the representations of different behaviors have high distinguishability and that the representations of the same one are semantically close. We detect anomalies by

<sup>1</sup>To simplify the expression, we omit the events related to motion sensor changes in these sequences.

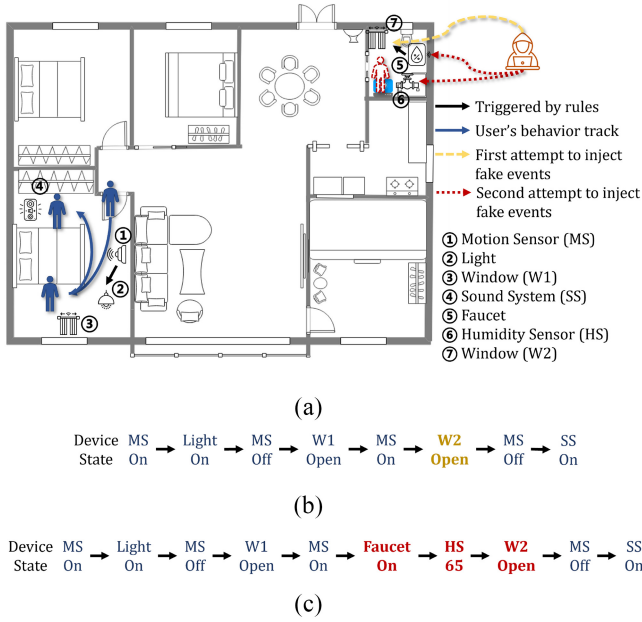


Fig. 1. Motivating example for the high-level behavior context in anomaly detection. (a) Smart IoT scenario and different attacks. (b) Event sequence for the first attack. (c) Event sequence for the second attack.

checking whether the testing behaviors belong to benign clusters.

- 3) We prototype our approach and conduct the evaluation on our self-developed hybrid smart IoT testbed. We configure 8 automation rules and 30 smart IoT devices and emulate a 7-day one-person living track on our self-developed testbed. The collected log has more than 8000 entries. Our cluster results show that there is a significant separation between the anomalous behaviors and the benign behaviors. The V-measure of our system behavior clustering results can reach 0.94. Compared to the existing methods, our approach increases precision and recall by 2.6 times and 9 times, respectively, validating the effectiveness of our approach.

## II. MOTIVATION AND PROBLEM ANALYSIS

Previous anomaly detection schemes [34], [35], [36] focused on the correlation between pairs of events. They predict the subsequent events that should occur based on preceding events or the preceding events based on the current event. These event-correlation-based anomaly detection methods can effectively detect traditional event injection attacks. We take Smart Home as an example scenario for the smart IoT system. For instance, in the smart home scenario shown in Fig. 1(a), we concentrate on smart home devices in the bedroom and bathroom areas, including the bedroom's motion sensor, light, window, and sound system, as well as the bathroom's faucet, humidity sensor, and window. We assume that an attacker can inject events or commands into the smart home platform through a vulnerability in the network protocol. In the first attack, the attacker injects the "bathroom window open" event [as the yellow dashed line shown in Fig. 1(a)]. At this time, assuming the user is active in the bedroom, their

behavior [as the blue solid line shown in Fig. 1(a)] includes entering the bedroom, passing the motion sensor, triggering an automation rule to turn on the light, closing the bedroom window, moving to the bedside, and turning on the sound system (passing the motion sensor again in the process). In this attack example, the event sequence collected by the system is shown in Fig. 1(b). The event-correlation-based anomaly detection method can predict that the ( $W2, Open$ ) event should not follow the ( $MS, On$ ) event, thereby detecting the attack. In practice, attackers will construct injected events more cleverly, using automation rules or physical environment correlations to affect the target device indirectly.

*Motivating Example#1—An Anomaly Occurred Within an Unrelated Behavior Context:* In the second attack shown in Fig. 1(a), the attacker injects a sequence of fake events consisting of "faucet turned on" and "humidity increased," creating a false impression that someone is in the bathroom [as the red dashed line shown in Fig. 1(a)]. If there is an automation rule in this smart home scenario that states "if the bathroom humidity rises, then turn on the bathroom fan to achieve the attack. The system event sequence corresponding to this attack example is shown in Fig. 1(c). The event-correlation-based anomaly detection method predicts that the ( $W2, Open$ ) event should follow the ( $HS, On$ ) event, and the log matches the prediction, failing to detect this type of attack.

To address the problem, current anomaly detection approaches [37], [38], [39] consider from a *high-level* behavioral context perspective. They summarize behavior contexts from historical patterns (i.e., sequences of events), build system behavior models, and predict subsequent events. For the attack shown in Fig. 1(a), even though the event sequence faucet turned on—humidity increased—window open seems reasonable, they lack the corresponding behavior context. Current behavior-context-based anomaly detection methods determine that the context where this event sequence currently exists is inconsistent with the context where it should exist, thereby detecting the attack.

*Motivating Example#2—Different Behavior Instances for the Same Behavior:* In practice, we observe that user behavior patterns in smart IoT systems are complex and diverse. Many similar (but not identical) event sequences (i.e., behavior instances) may represent the same user behavior. For the enter home behavior, after opening the door, users may perform some actions in the shoe cabinet area (opening or closing the cabinet door) and then close the door [as the blue line shown in Fig. 2(a)], or they may close the door first and then perform actions in the shoe cabinet area [as the yellow line shown in Fig. 2(a)]. Meanwhile, motion sensors near the door and the shoe cabinet will report movement detection. However, the order in which they report may vary depending on the specific sequence of the user's actions. Thus, a single behavior corresponds to multiple similar yet distinct event sequences. Fig. 2(b) and (c) illustrate the event sequences regarding different behavior instances, that is, the instances of the enter home behavior could be "door open—shoe cabinet

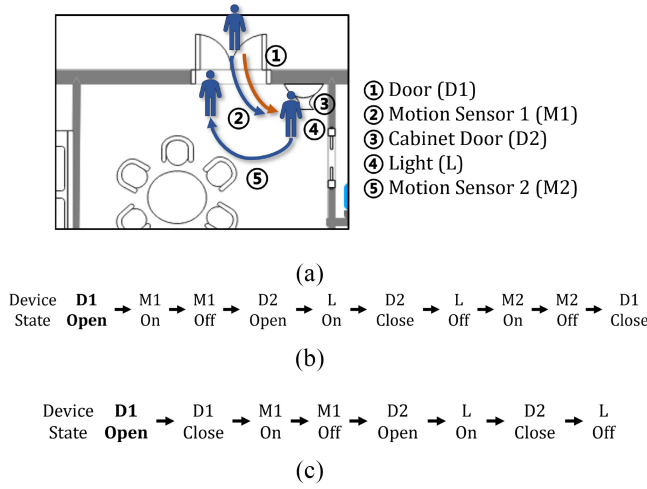


Fig. 2. Motivating example for critical/noncritical events in the enter home behavior. (a) User's different route for the enter home behavior. (b) Event sequence for the first behavior instance. (c) Event sequence for the second behavior instance.

open—light on—shoe cabinet close—light off—door close” and “door open—door close—shoe cabinet open—light on—shoe cabinet close—light off.”<sup>2</sup> We observe that within an event sequence, different events do not equally influence the semantics of a behavior. When assessing system behavior semantics, focusing on the context and order of *critical* events is crucial. For instance, in the enter home behavior, the door open event must occur first. The order of some noncritical events, or their presence or absence, should have less impact on the behavior's semantics.

Based on the above analysis, there exist two major challenges to accurately extract and summarize the semantics of system behaviors.

- 1) *Challenge 1 (Event Semantic Differentiation)*: Events in smart IoT systems record detailed device states but lack knowledge of high-level semantics critical for behavior representation. Specifically, two events of the same system entity could indicate different intentions in different scenarios. For example, the door open event could occur in both enter home, and “leave home” behavior. The semantics of a specific event should be inferred and represented based on its context.
- 2) *Challenge 2 (Behavior Semantic Summarization)*: Considering the second motivating example, different events in a system behavior instance impact its whole semantics differently. The semantics of system behaviors should rely more on critical events while tolerating variations in some noncritical events. Existing behavior-context-based approaches struggle to balance these two aspects, leading to false positives.

To address the first challenge, we represent system behaviors as behavior graphs and capture events' semantics based on their neighborhood relationship on the graph. For the second challenge, we use the frequency of events as their

<sup>2</sup>To simplify the expression, we omit the events related to motion sensor changes in these sequences.

TABLE I  
NOTATION LIST

| Symbol                        | Description                                                                                   |
|-------------------------------|-----------------------------------------------------------------------------------------------|
| $G$                           | A behavior graph                                                                              |
| $n$                           | The number of smart IoT device entities                                                       |
| $M \in \{0, 1\}^{n \times n}$ | The spatial state matrix                                                                      |
| $thr_t$                       | The time threshold for segmenting behaviors                                                   |
| $m$                           | The dimensionality of entity embedding vectors                                                |
| $\mathcal{V}$                 | The node set of behavior graphs                                                               |
| $\mathcal{A}$                 | The edge set of behavior graphs                                                               |
| $\tau = (h, r, t)$            | The basic unit of a behavior graph $G$ , where $h, t \in \mathcal{V}$ and $r \in \mathcal{A}$ |
| $\xi_i$                       | One-hot encoding for nodes and edges                                                          |
| $I_{max}$                     | Maximum number of intervals                                                                   |
| $w$                           | Selected initial window size                                                                  |
| $thr_c$                       | The similarity threshold for behavior clustering                                              |
| $thr_s$                       | The threshold for anomaly detection                                                           |

importance measurement when aggregating events' semantics as behaviors' semantics. We further propose a clustering approach based on the contextual semantics of the behavior graphs for anomaly detection.

### III. DESIGN

As shown in Fig. 3, our approach consists of three components: 1) *Behavior Graph Construction*; 2) *Behavior Classification*; and 3) *Anomaly Detection*. The workflow includes two phases: 1) *Training* and 2) *Testing*. We illustrate the *Training* phase as the arrowed solid lines and the *Testing* phase as the arrowed dotted lines in Fig. 3.

During the *Training* phase, the *Behavior Graph Construction* component takes the home layout and benign event log as inputs, utilizing the Behavior Extraction and Graph Construction modules to generate a Behavior Graph. Subsequently, the benign Behavior Graphs are fed into the *Behavior Classification* component, where they undergo Event Embedding, Behavior Summarization, and Behavior Clustering modules to classify behavior instances into benign behavior clusters.

In the *Testing* phase, our method employs the trained embedding model to perform Graph Construction and Behavior Summarization on the test event logs. Then, the testing behavior vectors are input into the *Anomaly Detection* component, which calculates the distance between the testing behavior vectors and the benign behavior clusters. Based on an anomaly distance threshold, suspicious behaviors that cannot be classified into benign behavior clusters are identified. These suspicious behaviors are validated manually to determine whether they are anomalies. If confirmed as anomalies, an alert is issued; if not, the behavior is considered a new benign behavior cluster and used to update the original benign behavior clusters. The notation list is shown in Table I.

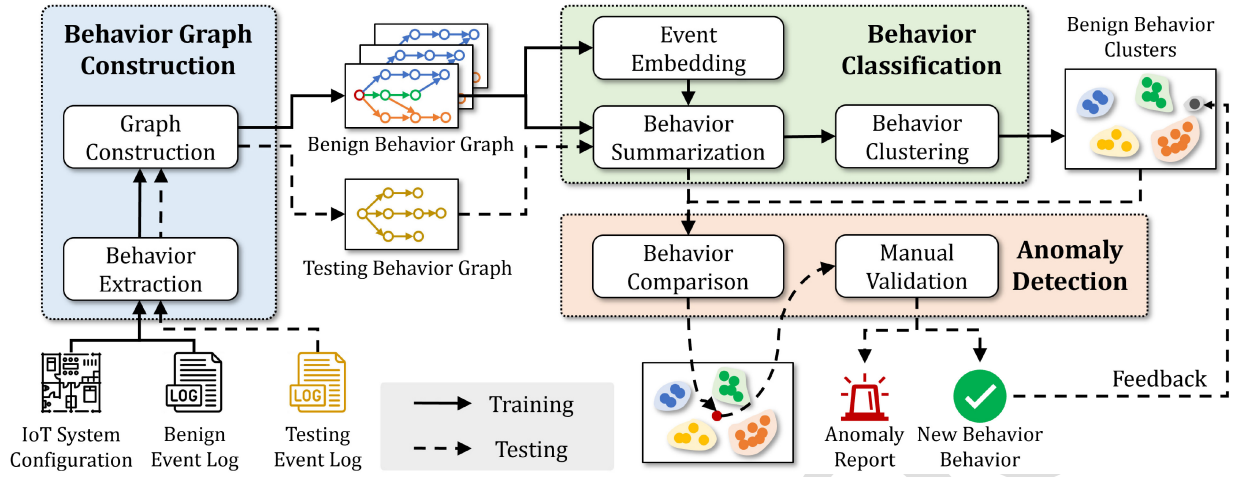


Fig. 3. Overall architecture of our approach.

### A. Behavior Graph Definition

We treat “the change of a device’s state” as an event and denote this kind of event as a triple  $e = (T, D, S)$ , indicating that at the time  $T$ , the state of the device  $D$  changes to  $S$ . We define a behavior as a combination of events from the event log  $E$  in a specific time duration. We denote a directed graph  $G = (\mathcal{V}, \mathcal{A}, \Phi)$  to describe a behavior, where  $\mathcal{V}$  and  $\mathcal{A}$  represent the node set and the directed edge set and  $\Phi : \mathcal{A} \rightarrow \{(u, v) | (u, v) \in \mathcal{V}^2, u \neq v\}$  is the incidence function mapping every edge to an ordered pair of nodes.

In our approach, the node  $v \in \mathcal{V}$  represents an event in this behavior, i.e.,  $v = (D, S)$ . For example, if there exists an event that a fan  $\text{fan}_1$  is turned on in this behavior, we will create a node  $v_i = (\text{fan}_1, \text{ON})$ . It is worth mentioning that for devices with discrete states, we retain their state values as  $S$  in the node  $v$ . In contrast, for devices with continuous states, we divide their possible value ranges into intervals based on certain principles and take the interval where their specific value falls as  $S$  in the node  $v$ .

The directed edge set  $\mathcal{A}$  includes all relationships between two nodes. Specifically, there are two kinds of relationships between nodes.

- 1) *Rule-triggered* relationship refers to a scenario where a subsequent event node  $v_j$  occurs due to a preceding event node  $v_i$  activating an automated rule. We define the edge connecting these two event nodes as a rule-triggered relationship, denoted by  $v_{ij} = \text{trigger\_}[event_j]$ . Here, the executed event ( $[event_j]$ ) is used to differentiate between various rule-triggered relationships. For instance, if there is an automation rule that states, “When the humidity sensor  $H1$  reads a value greater than 60, turn on the fan  $\text{Fan1}$ ,” then the events before and after the activation of this rule will be connected by an edge  $v_{ij} = \text{trigger\_turn\_on\_Fan1}$ .
- 2) *Temporally followed* relationship describes scenarios where changes in a device or sensor’s state result from user actions, such as turning on a television, or passive changes, such as a motion or humidity sensor detecting movement or increased moisture levels. Unlike the automated “rule-triggered” relationship, this type captures

interactions directly influenced by the user’s actions or environmental presence. Events in this relationship are sequentially connected by a temporally followed edge, denoted as  $v_{ij} = \text{followed\_}[event_j]$ , indicating the order in which  $[event_j]$  follows  $[event_i]$ . For instance, if a user activates a motion sensor by moving and then turns on the TV, these two events can be connected by an edge  $v_{ij} = \text{followed\_turn\_on\_TV}$ .

To enhance the semantic meaning of the graph, we place nodes with different environmental characteristics, such as physical actions, temperature, and humidity, on separate *channels*. For example, the physical action channel includes events like switching devices on or off or detecting motion, while the temperature channel captures changes in temperature sensors. The nodes in the same channel may have a rule-triggered or temporally followed relationship. In contrast, the nodes from the different channels can only be linked by the rule-triggered relationship. This connection shows how a change in one environmental characteristic can influence actions in another channel through automation rules.

Fig. 4(a) presents a partial smart IoT system layout configured with various smart IoT devices and multiple automation rules. In this bathroom scenario, the user engaged in walking and bathing activities. Fig. 4(b) shows parts of the log snippet related to the execution of Rule 1. We construct the behavior graph as follows. We first categorize events into physical actions, temperature, humidity, and time channels. In each channel, we connect events according to the time order of event occurrence with the “temporally followed” edge [as the black solid lines shown in Fig. 4(c)]. For example, we connect the nodes  $(L1, \text{ON})$  and  $(M1, \text{ON})$  with the temporally followed edge based on the first two lines in the log. Then, we consider the rule-triggered relationship. When the user enters the bathroom and turns on  $L1$ , he moves from  $M1$  to  $M2$ , which triggers Rule 1 and turns on  $L2$ . We connect the nodes  $(L1, \text{ON})$  and  $(L2, \text{ON})$  with the rule-triggered edge [as the blue dotted lines shown in Fig. 4(c)], as well as between  $(M2, \text{ON})$  and  $(L2, \text{ON})$ . In this way, we convert the log snippet in Fig. 4(b) into the shaded area in Fig. 4(c). Similarly, when the user takes a shower, it causes the humidity in the

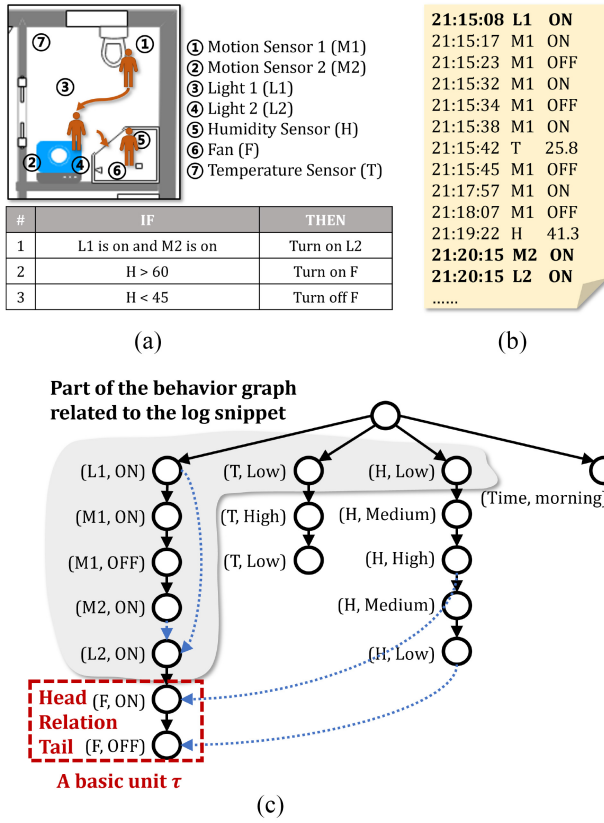


Fig. 4. Partial smart IoT scenario and the corresponding behavior graph of the user behavior occurring within it. (a) Room layout and the ruleset related to the example. (b) Log snippet of the example related to Rule 1. (c) Behavior graph of the example.

### Algorithm 1 Behavior Segmentation Algorithm

**Input:** Log file  $F$ , the time threshold for segmenting behaviors  $thr_t$ , the spatial state matrix  $M$ , selected initial window size  $w$

**Output:** Segmented behavior list  $B$

```

1: Initialize an empty list  $B$ 
2: while readline in  $F$  do
3:   Set current window  $W_i \leftarrow$  next  $w$  lines of log
4:   Find the main device  $dev_i$  in  $W_i$ 
5:   Remove unrelated devices using  $M$ 
6:   while Expand  $W_i$  by one line do
7:     if  $timedelta > thr_t$  then
8:       Save  $W_i$  to  $B$ 
9:       Continue
10:    end if
11:    if current device is not near  $dev_i$  then
12:      Expand  $W_i$  by  $w$  lines and find the main device
13:    if  $dev_i$  is near  $dev_j$  then
14:      Consider current line as noise
15:    else
16:      Save  $W_i$  to  $B$ 
17:      Continue
18:    end if
19:  else
20:    Continue
21:  end if
22: end while
23: end while
24: return  $B$ 

```

bathroom to rise, triggering Rule 2, and we connect the nodes (H1, High) and (F, ON) with the rule-triggered edge. After the user finishes and leaves the bathroom, the humidity decreases after a while, triggering Rule 3, and we connect the nodes (H1, Low) and (F, OFF) with the rule-triggered edge, too. Fig. 4(c) shows the complete behavior graph.

### B. Behavior Graph Construction

**Behavior Extraction:** Most anomaly detection methods use a fixed time window to segment a sequence of events. However, this approach has a significant limitation: the time window size can either be too large or too small, causing splitting large segments apart or merging small segments with others, which reduces detection accuracy. To address this, we develop a behavior segmentation algorithm that incorporates temporal and spatial information, effectively removing noise events, as illustrated in Algorithm 1.

The algorithm primarily uses smart IoT system logs as input to segment the logs into distinct behaviors. We utilize spatial information from the home layout and a time threshold to perform this segmentation. Specifically, we define a spatial state matrix  $M = \{m_{ij}\} \in \{0, 1\}^{n \times n}$ , where  $m_{ij} = 1$  if the  $i$ th device and the  $j$ th device are relatively close or in the same area, otherwise  $m_{ij} = 0$ , to represent the positional relationships between devices and segment events into different behaviors based on the spatial information and whether the time intervals

exceed a defined threshold (e.g., 10 min). Moreover, the spatial information can effectively handle noise events.

**Graph Construction:** We construct the behavior graph for each behavior based on the noise-reduced and segmented behavior log fragments. An intuitive insight is connecting related entities' state changes with edges. Following our definition, each device and its state is treated as a node. Without processing the continuous values, different states of the same device would correspond to distinct nodes, which would significantly increase the state space. We need to transform continuous state values (such as those from a temperature sensor) into a finite set of nodes, thereby reducing the size of the state space.

Some approaches divide numeric data into two parts, while others segment the data into multiple evenly spaced intervals. Segmenting with overly coarse or overly fine granularity can result in a high rate of false negatives or false positives in anomaly detection. We develop a more effective method for data partitioning considering the IoT system configuration and real-world state value distribution.

First, we check whether any numeric entities are part of a trigger in predefined rules. For example, if a rule specifies, "if temperature sensor exceeds 30 degrees, the air conditioner is turned on," then 30 becomes a reference point for segmenting the numeric data. Otherwise,

**Algorithm 2** Adaptive Interval Partitioning Algorithm

---

**Input:** All values  $V^d$  of a numerical device  $d$ , maximum number of intervals  $I_{\max}$

**Output:** The split point list  $SP^d$  for the device  $d$

- 1: Use  $V^d$  to compute the density distribution curve using KDE
- 2: Find all valley points on the curve as the initial split point list  $SP^d$
- 3: Initialize intervals using valley points
- 4: **while** number of intervals  $> I_{\max}$  **do**
- 5:   Find the two adjacent intervals with the most similar distribution
- 6:   Merge these two intervals in  $SP^d$
- 7: **end while**
- 8: **for** each pair of adjacent intervals **do**
- 9:   **if** the range of interval pair  $< thr_R$  and the mean density difference  $< thr_D$  **then**
- 10:     Merge these two intervals in  $SP^d$
- 11:   **end if**
- 12: **end for**
- 13: **return**  $SP^d$

---

we use kernel density estimation to determine the data distribution for each numeric entity and apply an adaptive interval partitioning algorithm to divide the entire data space into a variable number of intervals. The partitioning algorithm is illustrated in Algorithm 2.

We construct the behavior graph after obtaining the generalized log slices for each behavior. Specifically, we define a “start” node and then iteratively add nodes and edges based on the log entries. We place nodes with different environmental characteristics on separate channels and link the nodes in the same channel by the temporally followed relation. Then, we determine that if different nodes meet the requirements of the trigger and action in the automation rules, these nodes are connected by a rule-triggered relationship.

### C. Behavior Classification

**Event Embedding:** After representing each behavior as a graph, the key step is to embed these graphs into a continuous vector space. In our approach, we define the basic unit of the graph as a triple  $\tau = (h, r, t)$ , as shown in Fig. 4(c), where  $h$  (head) and  $t$  (tail) are nodes in the set of vertices  $\mathcal{V}$ , and  $r$  (relation) is an edge connecting  $h$  and  $t$ , belonging to the set of edges  $\mathcal{A}$ . Thus, a graph can be represented as a collection of such triples. We employ the TransE [41] model to embed these triples, which treats the relationships between entities as translations in the vector space. For a valid triple  $\tau = (h, r, t)$ , the model aims to make the formula  $h + r \approx t$  hold. Consider the semantic example: “The capital of France is Paris.” We can represent this relationship as the triple (*Paris*, *is the capital of*, *France*). TransE views the relation “*is the capital of*” as a translation that moves the vector representing “*Paris*” to the vector representing “*France*.” In other words, the vector for “*Paris*,” when translated by the vector for the relation “*is the capital of*,” should closely approximate the

vector for “*France*.” Similarly, in our scenario, consider a triple (*Humidity High*, *trigger\_turn\_on\_Fan*, *Fan ON*), TransE interprets the relation “*trigger\_turn\_on\_Fan*” as a translation that shifts the vector for “*Humidity High*” to the vector for “*Fan ON*.” This suggests that the TransE model is well-suited for graph embedding and has the potential to effectively capture the semantics of our triples.

During the embedding phase, we initialize all graph elements (nodes and edges) in the behavior graphs using one-hot encoding, representing each element as an  $m$ -dimensional binary vector  $\xi_i$ , where all components are 0 except for one, which is 1, and  $m$  is the total number of unique elements. For example, assuming two triples  $\tau_1 = (v_1, r_1, v_2)$  and  $\tau_2 = (v_2, r_2, v_3)$  collectively contain three unique nodes and two unique edges, such as  $\{v_1, v_2, v_3, r_1, r_2\}$ ,  $v_1$  would be encoded as  $(1, 0, 0, 0, 0)$ ,  $r_1$  as  $(0, 0, 0, 1, 0)$ , and so on. The head  $h$  of the triple  $\tau_1$  is  $v_1$ , so its encoding  $e_h$  is  $(1, 0, 0, 0, 0)$ , clearly identifying it as the first element in the set. The other elements’ encoding  $e_r$  and  $e_t$  of the triple  $\tau_1$  are  $(0, 0, 0, 1, 0)$  and  $(0, 1, 0, 0, 0)$ , respectively.

In the *Training* phase, TransE learns representations for entities and relationships by minimizing a margin-based ranking loss function. The goal is to ensure that for each triple  $(h, r, t)$  in graphs, the distance between the vectors  $e_h + e_r$  and  $e_t$  is minimized, while for each invalid triple  $(h', r, t')$ , the distance is maximized. In our experiments, we generate such invalid triples by randomly creating new triples and verifying that they do not exist in the graphs. The loss function is defined as

$$L = \sum_{(h,r,t) \in G} \sum_{(h',r',t') \notin G} (\|x_h + x_r - x_t\| - \|x_{h'} + x_{r'} - x_{t'}\| + \gamma) \quad (1)$$

where  $\|\cdot\|$  denotes the L1-norm function and  $\gamma$  is the margin hyperparameter. Note that a node’s embedding vector remains the same regardless of whether it appears as the head or the tail in a triple.

In summary, the input to the Event Embedding module consists of all nodes and edges present in the behavior graph, and the output is their corresponding embeddings, which are  $m$ -dimensional vectors for each graph element. As a result, a triple can be represented as a  $3m$ -dimensional vector.

**Behavior Summarization:** After event embedding, the next step is to combine the events to represent a behavior. Recall that each behavior graph consists of multiple triples, and each triple  $\tau = (h, r, t)$  has been represented as high-dimensional vectors. A straightforward method to obtain the semantic representation of a behavior instance is to sum the vectors of its triples. However, this approach assumes that all graph elements contribute equally to the behavior instance’s overall semantics. This assumption is often inaccurate because some nodes or edges contribute more significantly to the behavior than others. The key challenge is how to determine each element’s relative importance automatically. Our approach relies on the key insight that an event with a low probability of occurrence provides more information upon happening and is more significant in representing the system behavior semantics. Based on this, we propose using the frequency of nodes and edges to assess their importance. More specifically, we consider inverse

document frequency (IDF), a measure used in text analysis and information retrieval to quantify the importance of a term within a collection of documents [42], [43]. The formula is typically expressed as

$$\text{IDF}(t) = \log \frac{N}{n_t} \quad (2)$$

where in our context,  $t$  represents a specific node or edge,  $N$  denotes the total number of nodes or edges, and  $n_t$  indicates the number of the term  $t$ . Consequently, IDF effectively down-weights common terms and up-weights rarer ones that may be more significant in behaviors.

After weighting the graph elements, we derive the semantic representation of behavior instances by pooling their vectorized triples. We experiment with various pooling methods, such as addition, bi-interaction, and global average pooling. Ultimately, we choose addition pooling, which effectively captures the semantic information through simple summation.

**Behavior Clustering:** With the representation vectors of behaviors in hand, a natural next step is to explore how these behaviors can be grouped based on their similarities. Specifically, we introduce cosine similarity as a measure to determine the similarity between behavior vectors

$$S(F_m, F_n) = \frac{F_m \cdot F_n}{\|F_m\| \times \|F_n\|} = \frac{\sum_{e_i \in F_m} \sum_{e_j \in F_n} e_i \cdot e_j}{\sqrt{\sum_{e_i \in F_m} (e_i)^2} \times \sqrt{\sum_{e_j \in F_n} (e_j)^2}} \quad (3)$$

where  $F_m$  and  $F_n$  represent the vector of behavior instances. A higher cosine similarity indicates that the two behaviors are more similar.

Since the number of behavior categories is unknown, we use the agglomerative hierarchical clustering (AHC) method [44] to group all behavior instances. Starting from each behavior vector point as an independent cluster, we calculate the cosine similarity between each pair of clusters. Then, we find the two clusters with the closest cosine similarity; if the similarity between them is less than the threshold  $thr_c$ , we merge them into a new cluster and calculate the centroid of the new cluster. The centroid  $\mu_i$  of a cluster  $C_i$  is calculated as

$$\mu_i = \frac{1}{n_i} \sum_{j=1}^{n_i} e_j, e_j \in C_i \quad (4)$$

where  $e_j, j = 1, 2, \dots, n_i$  are the behavior instance vectors in the cluster  $C_i$ . Repeatedly, we calculate the similarities between the new cluster and the other clusters and perform the merging operation until there are no two clusters whose cosine similarity is less than the minimum similarity threshold  $thr_c$ .

In summary, in the *Behavior Classification* component, we embed events into a vector space to obtain the representation of behaviors, then cluster them based on their cosine similarities, resulting in several distinct benign behavior clusters.

#### D. Anomaly Detection

**Behavior Comparison:** After obtaining clusters of normal user behavior in the smart IoT system, a natural approach is to record the positions of these clusters in the embedding space.

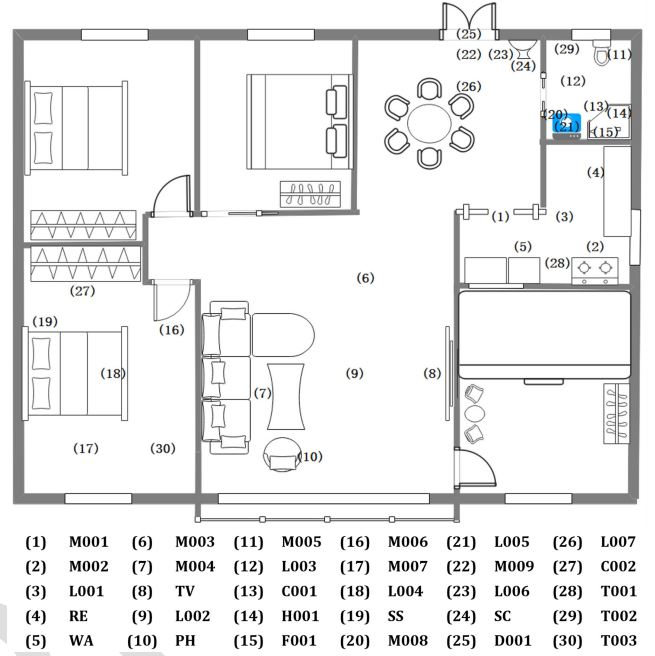


Fig. 5. Room layout and device deployment of the simulated smart IoT environment.

Any behavior instances that deviate significantly from all original benign cluster centroids will be considered anomalies. We take the testing log containing potential anomalies and use the same method to obtain the vector representation for each cluster. If the similarity between a testing behavior and every cluster centroid is below the anomaly detection threshold  $thr_d$ , we classify this behavior as suspicious.

**Manual Validation and Feedback:** These suspicious behaviors are validated manually to determine whether they are anomalies. If confirmed as anomalies, an alert is issued; if not, the behavior is considered a new benign behavior cluster and used to update the original benign behavior clusters.

## IV. IMPLEMENTATION AND EVALUATION

We prototype our approach and evaluate its performance within our custom-built smart IoT system, focusing on the effectiveness of behavior clustering and anomaly detection.

### A. Experiment Environment Configuration

Existing smart IoT system datasets seldom consider deploying automation rules, only reflecting user activity within the IoT environment. Although ARGUS collected a smart home dataset based on the configuration of automation rules, its dataset contains anomalous behaviors and lacks labels, making it unsuitable as a training set. Therefore, we need to collect our own smart IoT system dataset.

We establish a hybrid testbed designed to emulate a smart IoT environment. We choose a smart home environment as a representative scenario, featuring key areas, such as the kitchen, bedroom, bathroom, etc. The layout of our designed smart home environment is shown in Fig. 5. We also deploy 30 smart IoT devices in the environment, including

TABLE II  
RULESET CONFIGURATION OF THE SIMULATED  
SMART IoT ENVIRONMENT

| #  | Area          | IF                      | THEN                    |
|----|---------------|-------------------------|-------------------------|
| 1  | Kitchen       | M002 is on              | Turn on L001            |
| 2  |               | M002 is off             | Turn off L001 after 20s |
| 3  | Living Room   | PH is on and M003 is on | Turn on L002            |
| 4  | Bathroom      | H001 > 60               | Turn on F001            |
| 5  |               | H001 ≤ 45               | Turn off F001           |
| 6  | Bedroom       | It is 7:00              | Turn on SS              |
| 7  | Vanity        | M008 is on              | Turn on L005            |
| 8  |               | M008 is off             | Turn off L005 after 20s |
| 9  | Entrance hall | SC is on                | Turn on L006            |
| 10 |               | SC is off               | Turn off L006           |

nine motion sensors (M001–M009), seven lights (L001–L007), three temperature sensors (T001–T003), two contact sensors (C001 and C002), one humidity sensor (H001), one door (D001), one fan (F001), one refrigerator (RE), one dishwasher (WA), one television (TV), one phone (PH), one sound system (SS), and one shoe cabinet (SC). Based on the layout and the deployed devices, we configure 8 related automation rules, as shown in Table II.

This environment resembles a typical residential setting, enabling the realistic capture of everyday user behaviors. To populate the dataset, we predefine behaviors emphasizing logical and sequential patterns that monitor benign daily routines, such as bathing, watching TV, etc. Each behavior’s “patterns” are some sequence of device changes corresponding to the specific behavior. While the overall behavior design follows logical and plausible paths and sequences, we introduce randomness in how specific behaviors are performed, ensuring natural variations and spontaneity in their movements and interactions. We also add noise to their behaviors, enhancing the dataset’s representation of real-world scenarios. We simulate the behaviors of a single user over 7 days. Every day, the arrangement of user’s behaviors is different. After these behaviors are performed, the logs are exported from the Home Assistant platform, capturing detailed records of the users’ interactions and movements throughout the simulated environment. We finally generate more than 8000 log entries as the benign training dataset.

Regarding the abnormal dataset, we consider the generation of abnormal behavior from two perspectives.

- 1) Events that appear normal do not match their contextual semantics. For example, in a normal “bath” behavior, a user would sequentially enter the bathroom, open the shower door, and turn on the faucet, increasing humidity. However, an attacker might exploit SmartAPP to turn on the faucet at any time. In our simulation, we will inject the continuous event sequence of “faucet opened and humidity increase” at random moments, which is not in the normal environmental context.
- 2) The behavior significantly deviates from normal behavior. For example, for the “enter home” behavior, a user would typically open the door, change shoes, and then close the door. We will randomly add or delete events in the event sequence corresponding to this behavior,

such as removing the “open the door” event, causing the sequence to deviate significantly from normal behavior.

### B. Effectiveness of Behavior Clustering

We evaluate the effectiveness of clustering using the standard measures of clustering goodness: homogeneity  $h$ , completeness  $c$ , and their harmonic mean V-measure  $v$  [45], [46], [47]. These are scalar real-valued metrics in the  $[0, 1]$  range. In the context of our behavior clustering, homogeneity measures whether each cluster only contains the behavior instances of the same behavior, while completeness measures whether all instances of a specific behavior are assigned to the same cluster. The two metrics are computed based on the conditional entropy analysis. Specifically, let  $n$  denote the total number of behavior instances,  $n_b$  and  $n_k$  denote the number of behavior instances belonging to the behavior type  $b$  and the cluster  $k$ , and  $n_{b,k}$  denote the number of behavior instances from the behavior type  $b$  assigned to the cluster  $k$ . Then the entropy of the types  $H(B)$  is defined as

$$H(B) = - \sum_{b=1}^{|B|} \frac{n_b}{n} \log\left(\frac{n_b}{n}\right). \quad (5)$$

The conditional entropy of the types given the cluster assignments is defined as

$$H(B|K) = \sum_{k=1}^{|K|} P(K=k) H(B|K=k) = - \sum_{k=1}^{|K|} \sum_{b=1}^{|B|} \frac{n_{b,k}}{n} \log\left(\frac{n_{b,k}}{n}\right). \quad (6)$$

The entropy of the clusters ( $H(K)$ ) and the conditional entropy of clusters given type ( $H(K|B)$ ) are defined symmetrically.

For the homogeneity metric, the type distribution within each cluster should be skewed to a single type, that is, zero entropy. Rosenberg and Hirschberg [45] determined how close a given clustering is to this ideal by examining the conditional entropy of the type distribution given the proposed clustering  $H(B|K)$ . Considering that the raw conditional entropy is dependent on the size of the dataset and the distribution of type sizes, they normalized this value by the maximum reduction in entropy the clustering information could provide, specifically,  $H(B)$ . To adhere to the convention of 1 being desirable and 0 undesirable, they defined homogeneity as

$$h = 1 - \frac{H(B|K)}{H(B)}. \quad (7)$$

For the completeness metric, a clustering method must assign all data of a single type to a cluster. Similarly, they defined completeness as

$$c = 1 - \frac{H(K|B)}{H(K)}. \quad (8)$$

Furthermore, we use the harmonic mean of  $h$  and  $c$ , V-measure  $v$ , to penalize the imbalance between homogeneity and completeness and encourage more uniform clustering performance. The V-measure is defined as

$$v = 2 \times \frac{hc}{h+c}. \quad (9)$$

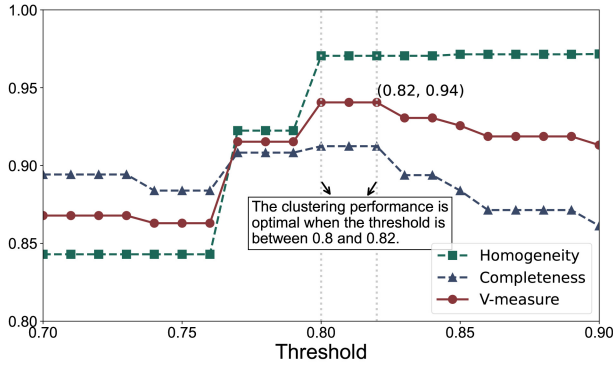


Fig. 6. Performance of the behavior summarization and clustering on different similarity thresholds.

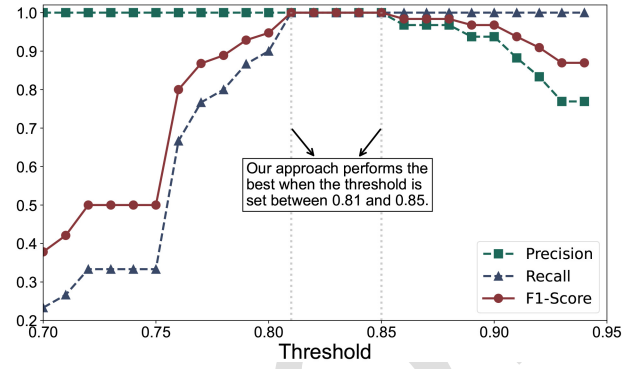


Fig. 8. Performance of the anomaly detection on different thresholds.

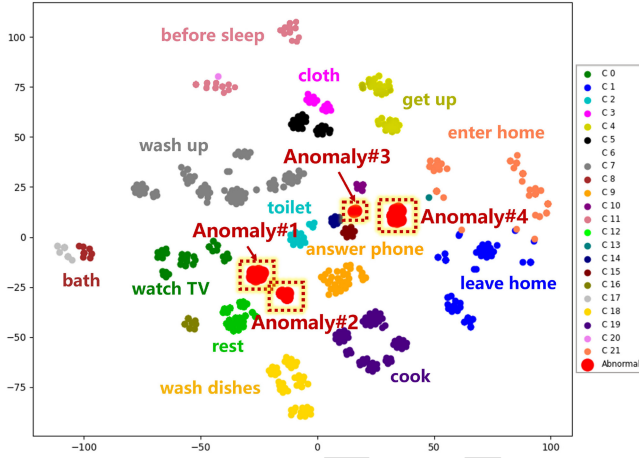


Fig. 7.  $t$ -SNE visualization of behavior clusters.

We use the precision  $P$ , recall  $R$ , and F1 score  $F1$  to evaluate the performance of our proposed model and baselines, which are defined as

$$P = \frac{TP}{TP + FP} \quad (10)$$

$$R = \frac{TP}{TP + FN} \quad (11)$$

$$F1 = \frac{2 \times P \times R}{P + R} \quad (12)$$

We apply a grid search to get the best threshold for anomaly detection. The threshold value is varied across a range from 0.7 to 0.95, with an incremental step size of 0.01. The precision, recall, and F1 scores of different thresholds are shown in Fig. 8. It can be deduced that when the threshold ranges between 0.81 and 0.85, both the precision and recall achieve a value of 1, indicating that within this range of threshold values, our anomaly detection method can accurately distinguish between benign behaviors and anomalous behaviors. For further experiments, we set the anomaly detection threshold to 0.85. Additionally, the embedded representations of anomalous behaviors are visualized in Fig. 7, as depicted in red. There is a significant separation between anomalous behaviors and the original normal behaviors, further validating the effectiveness of our method.

2) *Comparison With State of the Art (Compared With HAWatcher)*: HAWatcher [36] is a semantics-aware anomaly detection system for smart homes that leverages both event logs and device semantics to model normal behavior and identify deviations. It generates and refines the correlation between two events to represent the smart home's normal behaviors. However, according to our experiment, HAWatcher can detect almost no abnormal behaviors in our dataset.

We analyze the drawbacks from both the False Negative and False Positive perspectives. Regarding the False Negatives, since HAWatcher only captures the correlation between two events, attackers can craft the preceding events of anomalous ones in a sophisticated way to evade HAWatcher's detection, leading to missed alarms. As for the False Positives, due to user behavior diversity, some "noncritical" events do not always appear in different instances of the same behavior (i.e., event sequences). HAWatcher's deterministic relationships may flag behavior instances without these noncritical events as anomalies.

We apply a grid search to get the best similarity threshold for behavior clustering. The threshold value is varied across a range from 0.7 to 0.9, with an incremental step size of 0.01. The homogeneity, completeness, and V-measure of different thresholds is shown in Fig. 6. It can be deduced that when the threshold ranges between 0.8 and 0.82, the V-measure reaches its maximum value of 0.94, proving that our behavior summarization and clustering approach can effectively distinguish different behavior types within this range of threshold values.

Fig. 7 presents the visualization results of our behavior clustering on the benign dataset. Upon manual inspection, the resulting clusters correspond to our predefined behaviors. This experiment demonstrates that our extraction and embedding of behaviors can effectively represent the semantics of behaviors, with similar behaviors being semantically close and high distinguishability between different behavioral semantics.

### C. Effectiveness of Anomaly Detection

1) *Selection of Anomaly Decision Threshold*: As described in Section III-D, during anomaly detection, we represent the logs containing anomalous behaviors using the same method and employ a cosine similarity threshold to identify these anomalies. We first evaluate the choice of threshold effectiveness for anomaly detection.

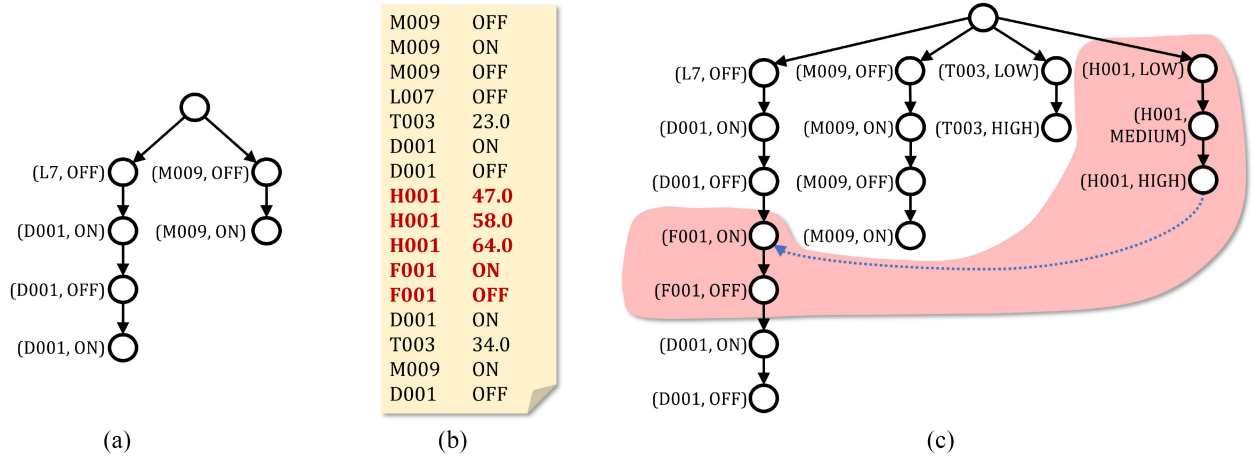


Fig. 9. Comparison between the benign behavior and Case#1. (a) Behavior graph of leave home. (b) Log snippet of case 1. (c) Behavior graph of Case#1.

TABLE III  
COMPARISON RESULTS BETWEEN OUR APPROACH AND ARGUS

|              | Precision | Recall | F1-score |
|--------------|-----------|--------|----------|
| ARGUS        | 27.3%     | 10%    | 14.6%    |
| Our approach | 100%      | 100%   | 100%     |

Compared With ARGUS: ARGUS [39] is a self-learning intrusion detection system for detecting contextual attacks on IoT environments. It employs an unsupervised Deep Neural Network to learn and model typical device behaviors based on their context, enabling the detection of known and novel attacks through monitoring the state and actions of IoT devices. The performance of AUGUS on our dataset is shown in Table III. It can be observed that AUGUS's performance is not satisfactory, while our approach increases precision and recall by 2.6 times and 9 times, respectively. We provide a detailed explanation through two case studies.

**Case#1—An Anomaly Occurred Within an Unrelated Behavior Context (False Negative):** Fig. 9(a) illustrates a representative behavior graph of leave home. In our experiment, we insert an anomaly when the user leaves home. Fig. 9(b) presents a log snippet corresponding to Case#1. In the bold red part, the bathroom fan is turned on as the humidity rises, which appears to be strongly correlated. However, the event sequences before and after this segment are user behaviors in the entrance hall, lacking relevant context. This segment should, therefore, be considered anomalous behavior, indicating that an attacker may have compromised the device. Our approach generates its behavior graph as shown in Fig. 9(c), and decides that this behavior instance will not fall into the cluster corresponding to the leave home behavior. However, ARGUS failed to detect this anomaly, which reveals two problems with ARGUS. First, the deep learning model used by ARGUS requires a great amount of data to learn behavioral patterns effectively. Second, ARGUS can only predict event sequences within a fixed time window; thus, for this case, ARGUS naturally separates the anomalous event sequence

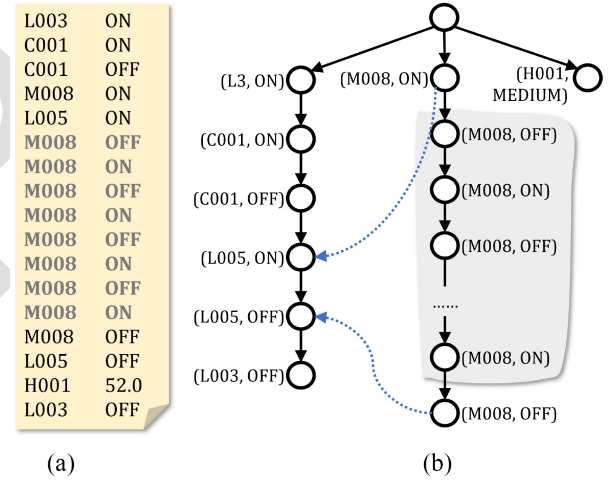


Fig. 10. Log snippet and the behavior graph of Case#2. (a) Log snippet of Case#2. (b) Behavior graph of Case#2.

from the surrounding ones, treating them as distinct behavioral instances.

**Case#2—Frequent Sensor Value Changes in a Benign Behavior (False Positive):** In the smart home environment, sensor changes also affect the semantics of behaviors. However, the frequency of sensor changes can fluctuate significantly, as the gray log entries are shown in Fig. 10(a). In our approach to graph construction and behavior representation, we measure the importance of different events to the semantics of behaviors, using this to weight and aggregate event semantics to form behavior semantics, thereby reducing the impact of fluctuations in the number of sensor changes [i.e., the gray part in Fig. 10(b)]. However, ARGUS cannot handle this kind of “noise” with semantics and incorrectly flags it as an anomaly.

## V. RELATED WORK

Anomaly detection is essential in smart IoT systems for maintaining security, efficiency, and reliability. Identifying unusual patterns in device and sensor data enables early

TABLE IV  
ANALYSIS OF EXISTING ANOMALY DETECTION APPROACHES

| Work                  | Methods               | Source <sup>a</sup> |   |   | Model Granularity <sup>b</sup> |   | Order Insensitivity | Contextual Features <sup>c</sup> |     |
|-----------------------|-----------------------|---------------------|---|---|--------------------------------|---|---------------------|----------------------------------|-----|
|                       |                       | N                   | P | L | E                              | B |                     | Dev                              | Env |
| CLEAN [34]            | Statistics            |                     |   | ✓ | ✓                              |   | ✓                   |                                  |     |
| DICE [35]             | Statistics            |                     |   | ✓ | ✓                              |   |                     | ✓                                |     |
| Dilraj et al. [48]    | OCSVM                 |                     | ✓ |   | ✓                              |   | N/A                 |                                  |     |
| Alsabilah et al. [49] | Kalman Filter         |                     | ✓ |   | ✓                              |   | N/A                 |                                  |     |
| Hodo et al. [50]      | Multilayer Perceptron | ✓                   |   |   | ✓                              |   | N/A                 |                                  |     |
| CADeSH [38]           | Autoencoder           | ✓                   |   |   | ✓                              |   | N/A                 |                                  |     |
| Novák et al. [51]     | Self-Organizing Map   |                     |   | ✓ |                                | ✓ | ✓                   |                                  | ✓   |
| Fahad et al. [52]     | H2O Autoencoder       |                     |   | ✓ |                                | ✓ | ✓                   |                                  | ✓   |
| Alaghbari et al. [53] | OCD-AE                |                     |   | ✓ |                                | ✓ | ✓                   |                                  | ✓   |
| Aegis [37]            | Markov Chain          |                     |   | ✓ |                                | ✓ |                     |                                  |     |
| Hela et al. [54]      | Markov Logic Network  |                     |   | ✓ |                                | ✓ |                     | ✓                                |     |
| Fahad et al. [55]     | Statistics            |                     |   | ✓ |                                | ✓ | ✓                   |                                  | ✓   |
| Yamauchi et al. [56]  | Tree Models           |                     |   | ✓ |                                | ✓ |                     | ✓                                | ✓   |
| HAWatcher [36]        | NLP, HT               |                     |   | ✓ | ✓                              |   |                     | ✓                                | ✓   |
| ARGUS [39]            | GRU-based AE          |                     |   | ✓ |                                | ✓ |                     | ✓                                |     |
| <b>Our approach</b>   | TransE, AHC           |                     |   | ✓ |                                | ✓ | ✓                   | ✓                                | ✓   |

<sup>a</sup>N - Network traffic, P - Power consumption, L - Log file

<sup>b</sup>E - Event, B - Behavior

<sup>c</sup>Dev - Device, Env - Environment

detection of issues like equipment malfunctions or security breaches, ensuring safer and more efficient smart IoT systems.

Some works utilize edge-side sensor data, power consumption metrics, or network packets for anomaly detection. For example, CLEAN [34] is a knowledge-driven approach for detecting anomalies in event-driven binary sensors. It combines knowledge and statistical models to detect random and systematic sensor anomalies. Moreover, DICE [35] is an automated solution for detecting and identifying faulty devices through context extraction. It leverages sensor correlations and state transitions to detect failures rapidly. Dilraj et al. [48] utilized one-class SVM to detect anomalies in IoT devices based on power consumption. Alsabilah and Rawat [49] modeled a smart home system using a Kalman filter to estimate the system's state based on energy consumption data from smart meters and apply the Shapiro–Wilk test to assess the fit between the estimated and measured data, triggering alerts when necessary. Hodo et al. [50] employed multilayer perceptron trained on Internet packet traces to classify normal and threat patterns in IoT networks, demonstrating high effectiveness in detecting and mitigating DDoS/DoS attacks. CADeSH [38] uses Autoencoder to differentiate between frequent and infrequent network traffic, then applies clustering methods to classify the infrequent traffic into known and unknown categories for anomaly detection. These methods observe anomalies in smart IoT systems from a limited, localized perspective and cannot infer behaviors based on contextual information surrounding events.

Other research focuses on learning normal user behavior patterns from smart IoT system logs to detect irregularities. Novák et al. [51] used Self-Organizing Maps to learn users' daily activities to identify unusual behaviors, focusing on abnormal short- or long-term activities occurring in atypical locations. Fahad and Tahir [52] utilized the Probabilistic Neural Network, while Alaghbari et al. [53] applied the

Overcomplete Deep Autoencoder, both for detecting anomalies in elderly individuals' daily home activities. Sikder et al. [37] detected malicious behavior using Markov chain-based machine learning and alerts users while adapting its model based on user response to enhance accuracy through adaptive training. Hela et al. [54] employed causal association rule to extract the causes of anomalies from a given dataset, which are then used in real-time analysis to detect anomaly risks with a Markov Logic Network. Fahad and Rajarajan [55] proposed a statistical method that analyses anomalies using DBSCAN, leveraging the number of events and the duration of activity execution from the identified instances.

Furthermore, some studies incorporate additional semantic information to more accurately guide anomaly detection. Yamauchi et al. [56] modeled user behavior using a tree of event sequences, where conditions are based on a combination of time and sensor readings. Anomalies are identified when behaviors do not satisfy these conditions or are absent from the event sequence tree. HAWatcher [36] employs NLP techniques to generate hypothesized correlations based on semantic relationships, which are then validated against log data. Any events in the logs that deviate from these simulated correlations are flagged as anomalies. ARGUS [39] adopts a GRU-based autoencoder to evaluate device actions within their contextual setting, detecting stealthy control plane attacks in IoT networks. Table IV summarizes all the works mentioned above.

Compared to previous works, which cannot often infer behaviors within a broader context or rely solely on statistical or knowledge-driven methods, our approach offers a more comprehensive understanding of user behavior by integrating both spatial and temporal factors. By incorporating device locations, behavior segmentation, and embedding-based modeling, our method enables more accurate and context-aware anomaly detection, particularly in complex smart home

environments where behaviors are influenced by both time and space. Furthermore, using clusters to characterize normal behavior patterns provides a more adaptive and scalable solution, making our approach more robust to user routine and environmental setting variations.

## VI. CONCLUSION

In this article, we present an anomaly detection approach for smart IoT systems through the contextual semantics of behavior graphs. We first design a graph-based representation to capture the relationships among events in system behavior, generating graph representations of benign behavior instances from event logs. We create a semantic representation that captures key behaviors while tolerating minor variations by assigning importance weights to events. This representation reduces environmental noise and enhances semantic clarity, grouping behaviors into distinct clusters. Anomalies are detected when behaviors deviate from these benign clusters. Our evaluation on a smart IoT testbed with over 8000 log entries demonstrates the effectiveness and scalability of our approach in distinguishing normal and anomalous behaviors, offering a robust solution for securing smart IoT environments.

## REFERENCES

- [1] G. Ho, D. Leung, P. Mishra, A. Hosseini, D. Song, and D. Wagner, "Smart locks: Lessons for securing commodity Internet of Things devices," in *Proc. 11th ACM Asia Conf. Comput. Commun. Security*, 2016, pp. 461–472.
- [2] E. Ronen and A. Shamir, "Extended functionality attacks on IoT devices: The case of smart lights," in *Proc. IEEE Eur. Symp. Security Privacy*, 2016, pp. 3–12.
- [3] C. Koliadis, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, Jul. 2017.
- [4] G. Zhang, C. Yan, X. Ji, T. Zhang, T. Zhang, and W. Xu, "DolphinAttack: Inaudible voice commands," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2017, pp. 103–117.
- [5] Y. Liang, Z. Cai, J. Yu, Q. Han, and Y. Li, "Deep learning based inference of private information using embedded sensors in smart devices," *IEEE Netw.*, vol. 32, no. 4, pp. 8–14, Jul./Aug. 2018.
- [6] X. Wang, Y. Sun, S. Nanda, and X. Wang, "Looking from the mirror: Evaluating IoT device security through mobile companion apps," in *Proc. 28th USENIX Security Symp.*, 2019, pp. 1151–1167.
- [7] F. Meneghello, M. Calore, D. Zucchetto, M. Polese, and A. Zanella, "IoT: Internet of threats? A survey of practical security vulnerabilities in real IoT devices," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8182–8201, Oct. 2019.
- [8] J. Mao, Z. Liu, Q. Lin, and Z. Liang, "Semantic-fuzzing-based empirical analysis of voice assistant systems of asian symbol languages," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9151–9166, Jun. 2022.
- [9] A. Bhardwaj, K. Kaushik, M. Alshehri, A. A.-B. Mohamed, and I. Keshta, "ISF: Security analysis and assessment of smart home IoT-based firmware," *ACM Trans. Sens. Netw.*, to be published.
- [10] S. Vetrivel, V. Van Harten, C. H. Gañán, M. Van Eeten, and S. Parkin, "Examining consumer reviews to understand security and privacy issues in the market of smart home devices," in *Proc. 32nd USENIX Security Symp.*, 2023, pp. 1523–1540.
- [11] B. Fouladi and S. Ghanoun. "Honey, I'm home!! Hacking z-wave home automation systems: Black Hat." 2013. [Online.] Available: <https://cybergibbons.com/wp-content/uploads/2014/11/honeyimhome-131001042426-phpapp01.pdf>
- [12] E. Ronen, A. Shamir, A.-O. Weingarten, and C. O'Flynn, "IoT goes nuclear: Creating a ZigBee chain reaction," in *Proc. IEEE Symp. Security Privacy (SP)*, 2017, pp. 195–212.
- [13] E. Fernandes, A. Rahmati, J. Jung, and A. Prakash, "Decentralized action integrity for trigger-action IoT platforms," in *Proc. Netw. Distrib. Syst. Security Symp.*, 2018, pp. 1–16.
- [14] Y. Jia et al., "Burglars' IoT paradise: Understanding and mitigating security risks of general messaging protocols on IoT clouds," in *Proc. IEEE Symp. Security Privacy (SP)*, 2020, pp. 465–481.
- [15] Q. Wang et al., "MPIInspector: A systematic and automatic approach for evaluating the security of IoT Messaging protocols," in *Proc. 30th USENIX Security Symp.*, 2021, pp. 4205–4222.
- [16] B. Yuan et al., "MQTTactic: Security analysis and verification for logic flaws in MQTT implementations," in *Proc. IEEE Symp. Security Privacy (SP)*, 2024, pp. 2385–2403.
- [17] E. Fernandes, J. Jung, and A. Prakash, "Security analysis of emerging smart home applications," in *Proc. IEEE Symp. Security Privacy (SP)*, 2016, pp. 636–654.
- [18] X. Zheng, Z. Cai, and Y. Li, "Data linkage in smart Internet of Things systems: A consideration from a privacy perspective," *IEEE Commun. Mag.*, vol. 56, no. 9, pp. 55–61, Sep. 2018.
- [19] B. Yuan, Y. Jia, L. Xing, D. Zhao, X. Wang, and Y. Zhang, "Shattered chain of trust: Understanding security risks in cross-cloud IoT access delegation," in *Proc. 29th USENIX Security Symp.*, 2020, pp. 1183–1200.
- [20] M. O. Ozmen, R. Song, H. Farrukh, and Z. B. Celik, "Evasion attacks and defenses on smart home physical event verification," in *Proc. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2023, pp. 1–18.
- [21] R. Pickren, T. Shekari, S. Zonouz, and R. Beyah, "Compromising industrial processes using Web-based programmable logic controller malware," in *Proc. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2024, pp. 1–18.
- [22] X. Cheng et al., "Meta computing," *IEEE Netw.*, vol. 38, no. 2, pp. 225–231, Mar. 2024.
- [23] M. Xu, Y. Guo, Q. Hu, Z. Xiong, D. Yu, and X. Cheng, "A trustless architecture of blockchain-enabled metaverse," *High-Confid. Comput.*, vol. 3, no. 1, 2023, Art. no. 100088.
- [24] Y. J. Jia et al., "ContextIoT: Towards providing contextual integrity to appified IoT platforms," in *Proc. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2017, pp. 1–15.
- [25] Z. Cai and X. Zheng, "A private and efficient mechanism for data uploading in smart cyber-physical systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 2, pp. 766–775, Apr.–Jun. 2020.
- [26] W. He et al., "Rethinking access control and authentication for the home Internet of Things (IoT)," in *Proc. 27th USENIX Security Symp.*, 2018, pp. 255–272.
- [27] H. Chi, Q. Zeng, X. Du, and L. Luo, "PFirewall: Semantics-aware customizable data flow control for smart home privacy protection," in *Proc. Netw. Distrib. Syst. Security (NDSS) Symp.*, 2021, pp. 1–18.
- [28] S. Zhu, R. Li, Z. Cai, D. Kim, D. Seo, and W. Li, "Secure verifiable aggregation for blockchain-based federated averaging," *High-Confid. Comput.*, vol. 2, no. 1, 2022, Art. no. 100046.
- [29] Y. Guo, Z. Wan, and X. Cheng, "When blockchain meets smart grids: A comprehensive survey," *High-Confid. Comput.*, vol. 2, no. 2, 2022, Art. no. 100059.
- [30] Z. Yang et al., "An attribute-based access control scheme using blockchain technology for IoT data protection," *High-Confid. Comput.*, vol. 4, no. 3, 2024, Art. no. 100199.
- [31] Z. B. Celik, G. Tan, and P. D. McDaniel, "IoTGuard: Dynamic enforcement of security and safety policy in commodity IoT," in *Proc. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2019, pp. 1–15.
- [32] H. Chi, Q. Zeng, and X. Du, "Detecting and handling IoT interaction threats in multi-platform multi-control-channel smart homes," in *Proc. 32nd USENIX Security Symp.*, 2023, pp. 1559–1576.
- [33] Y. Yu, Y. Xu, K. Huang, and J. Liu, "TAPFixer: Automatic detection and repair of home automation vulnerabilities based on negated-property reasoning," in *Proc. 33rd USENIX Security Symp.*, 2024, pp. 4945–4962.
- [34] J. Ye, G. Stevenson, and S. Dobson, "Fault detection for binary sensors in smart home environments," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. (PerCom)*, 2015, pp. 20–28.
- [35] J. Choi et al., "Detecting and identifying faulty IoT devices in smart home with context extraction," in *Proc. 48th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, 2018, pp. 610–621.
- [36] C. Fu, Q. Zeng, and X. Du, "HAWatcher: Semantics-aware anomaly detection for appified smart homes," in *Proc. 30th USENIX Security Symp.*, 2021, pp. 4223–4240.
- [37] A. K. Sikder, L. Babun, H. Aksu, and A. S. Uluagac, "Aegis: A context-aware security framework for smart home systems," in *Proc. 35th Annu. Comput. Security Appl. Conf.*, 2019, pp. 28–41.
- [38] Y. Meidan, D. Avraham, H. Libhaber, and A. Shabtai, "CADESH: Collaborative anomaly detection for smart homes," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8514–8532, May 2023.

- [39] P. Rieger, M. Chilese, R. Mohamed, M. Miettinen, H. Fereidooni, and A.-R. Sadeghi, "ARGUS: Context-based detection of stealthy IoT infiltration attacks," in *Proc. 32nd USENIX Security Symp.*, 2023, pp. 4301–4318.
- [40] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948.
- [41] A. Bordes, N. Usunier, A. Garcia-Durán, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *Proc. 27th Int. Conf. Neural Inf. Process. Syst.*, vol. 26, 2013, pp. 2787–2795.
- [42] H. Schütze, C. D. Manning, and P. Raghavan, *Introduction to Information Retrieval*, vol. 39. Cambridge, U.K.: Cambridge Univ. Press, 2008.
- [43] J. Zeng, Z. L. Chua, Y. Chen, K. Ji, Z. Liang, and J. Mao, "WATSON: Abstracting behaviors from audit logs via aggregation of contextual semantics," in *Proc. NDSS*, 2021, pp. 1–18.
- [44] F. Murtagh and P. Contreras, "Algorithms for hierarchical clustering: An overview," *Wiley Interdiscipl. Rev., Data Min. Knowl. Discov.*, vol. 2, no. 1, pp. 86–97, 2012.
- [45] A. Rosenberg and J. Hirschberg, "V-measure: A conditional entropy-based external cluster evaluation measure," in *Proc. Joint Conf. Empir. Methods Nat. Lang. Process. Comput. Nat. Lang. Learn. (EMNLP-CoNLL)*, 2007, pp. 410–420.
- [46] Y. Ye, Z. Zhang, F. Wang, X. Zhang, and D. Xu, "NetPlier: Probabilistic network protocol reverse engineering from message traces," in *Proc. NDSS*, 2021, pp. 1–18.
- [47] W. Pang, S. Panda, J. Amjad, C. Diot, and R. Govindan, "CloudCluster: Unearthing the functional structure of a cloud service," in *Proc. 19th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2022, pp. 1213–1230.
- [48] M. Dilraj, K. Nimmy, and S. Sankaran, "Towards behavioral profiling based anomaly detection for smart homes," in *Proc. IEEE Region Conf. (TENCON)*, 2019, pp. 1258–1263.
- [49] N. Alsabilah and D. B. Rawat, "Anomaly detection in smart home networks using Kalman filter," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2021, pp. 1–6.
- [50] E. Hodo et al., "Threat analysis of IoT networks using artificial neural network intrusion detection system," in *Proc. Int. Symp. Netw., Comput. Commun. (ISNCC)*, 2016, pp. 1–6.
- [51] M. Novák, F. Jakab, and L. Lain, "Anomaly detection in user daily patterns in smart-home environment," *J. Sel. Areas Health Inform.*, vol. 3, no. 6, pp. 1–11, 2013.
- [52] L. G. Fahad and S. F. Tahir, "Activity recognition and anomaly detection in smart homes," *Neurocomputing*, vol. 423, pp. 362–372, Jan. 2021.
- [53] K. A. Alaghbari, M. H. M. Saad, A. Hussain, and M. R. Alam, "Activities recognition, anomaly detection and next activity prediction based on neural networks in smart homes," *IEEE Access*, vol. 10, pp. 28219–28232, 2022.
- [54] S. Hela, B. Amel, and R. Badran, "Early anomaly detection in smart home: A causal association rule-based approach," *Artif. Intell. Med.*, vol. 91, pp. 57–71, Sep. 2018.
- [55] L. G. Fahad and M. Rajarajan, "Anomalies detection in smart-home activities," in *Proc. IEEE 14th Int. Conf. Mach. Learn. Appl. (ICMLA)*, 2015, pp. 419–422.
- [56] M. Yamauchi, Y. Ohsita, M. Murata, K. Ueda, and Y. Kato, "Anomaly detection in smart home operation from user behaviors and home conditions," *IEEE Trans. Consum. Electron.*, vol. 66, no. 2, pp. 183–192, May 2020.



**Shuyuan Chang** received the B.S. degree from Beihang University, Beijing, China, in 2023, where he is currently pursuing the M.S. degree with the School of Cyber Science and Technology. His research interests include IoT security, ICS security, and Web security.



**Jian Mao** (Member, IEEE) received the B.S. and Ph.D. degrees from Xidian University, Xi'an, Shaanxi, China, in 1997 and 2004, respectively. She is a Professor with the School of Cyber Science and Technology, Beihang University, Beijing, China. Her research interests include IoT security, Web security, and mobile security.



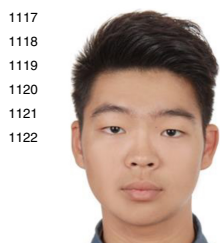
**Qiange Liu** received the B.S. degree from Sichuan University, Chengdu, Sichuan, China in 2019. She is currently pursuing the Ph.D. degree with the School of Cyber Science and Technology, Beihang University, Beijing, China. Her research interests include IoT security and Web security.



**Ziwen Liu** received the B.S. degree from Beihang University, Beijing, China, in 2020, where she is currently pursuing the Ph.D. degree with the School of Cyber Science and Technology. Her research interests include IoT security, SDN security, and Web security.



**Yifan Lu** received the B.E. degree from Beihang University, Beijing, China, in 2023, where she is currently pursuing the M.S. degree with the School of Cyber Science and Technology. Her research interests include IoT security and Web security.



**Qixiao Lin** received the B.S. degree from Beihang University, Beijing, China, in 2018, where he is currently pursuing the Ph.D. degree with the School of Cyber Science and Technology. His research interests include IoT security, mobile security, and Web security.



**Yan Huo** (Senior Member, IEEE) received the B.E. and Ph.D. degrees from Beijing Jiaotong University, Beijing, China, in 2004 and 2009, respectively. He is currently a Professor with the School of Electronics and Information Engineering, Beijing Jiaotong University. His current research focuses on wireless communications, security and privacy, and Internet of Things.